

Projeto 1 - Processamento de Imagens em C com SDL3

Etapa 2 - Análise final e implementação

Universidade Presbiteriana Mackenzie, Faculdade de Computação e Informática
Curso: Ciência da Computação · **Disciplina:** Computação Visual **Professor:** André Kishimoto · **Semestre:** 2026.2 **Data:** 20/09/2026

1. Integrantes do grupo

Nome completo	RA
João Guilherme Messias de Oliveira Santos	10426110
Luis Felipe Cunha	10419514

2. Link do repositório público

<https://github.com/Joaogmos/computacao-visual-projeto1>

3. Fonte usada nos textos do programa

A família escolhida foi a DejaVu Sans (arquivo `DejaVuSans.ttf`, embutido em `assets/fonts/`). Já tínhamos cogitado essa fonte desde o relatório da Etapa 1 (seção 7.5),

então não houve mudança de plano aqui.

A razão principal é a licença: a DejaVu Sans deriva da Bitstream Vera Fonts e usa uma licença permissiva que autoriza redistribuir, modificar e embarcar o arquivo em outros projetos, desde que o aviso de copyright seja mantido (o texto completo está em `assets/fonts/LICENSE.txt`). Isso evita depender de fontes instaladas no sistema operacional, que variam entre Windows e Linux, ou de uma fonte cuja licença não deixe redistribuir o arquivo junto do código. A DejaVu Sans também cobre bem os caracteres que o programa precisa: acentuação em português, números, símbolos dos botões.

O carregamento acontece em `src/text.c`, na função `text_load_font()`, que monta o caminho `<diretório do executável>/assets/fonts/DejaVuSans.ttf` usando `SDL_GetBasePath()` em vez de um caminho fixo ou de fontes do sistema. Isso garante que a fonte carregue corretamente em qualquer sistema operacional, desde que a pasta `assets/` esteja junto do executável, o que o `Makefile` já cuida de fazer automaticamente (alvo `assets`).

4. Refatoração do código-fonte original

Esse item não se aplica do jeito que o enunciado normalmente pressupõe. Não tivemos acesso a um código-base específico do repositório da disciplina na hora de implementar: esse repositório de exemplos fica no Moodle e ninguém do grupo chegou a localizá-lo ou baixá-lo até esta etapa. Diante disso, a decisão (já registrada como próximo passo lá na Etapa 1) foi implementar o programa inteiro do zero, seguindo à risca o "Escopo e funcionalidades obrigatórias" do enunciado e a documentação oficial da SDL3/SDL3_image/SDL3_ttf, em vez de partir de um código-exemplo específico.

Não existe, portanto, uma refatoração de código de terceiros para descrever. O que existe é uma decisão de arquitetura tomada desde o início: dividir a responsabilidade em módulos pequenos (`image`, `histogram`, `button`, `text`, `app`, `main`) em vez de concentrar tudo num arquivo só, para que cada requisito do enunciado correspondesse a uma unidade de código isolada e testável. O critério de "qualidade do código" do enunciado pede responsabilidades claras, e um código de exemplo único costuma vir mais monolítico que isso.

5. Problemas encontrados durante o desenvolvimento e soluções

API de janela filha (`SDL_CreateWindowWithProperties`). Como já antecipávamos na Etapa 1 (seção 7.1), a documentação da SDL3 descreve as propriedades

(`SDL_PROP_WINDOW_CREATE_PARENT_POINTER` , `_X_NUMBER` , `_Y_NUMBER` , `_WIDTH_NUMBER` , `_HEIGHT_NUMBER` , `_TITLE_STRING`) mas não deixa claro, num lugar só, como montar a chamada inteira.

Resolvemos lendo o cabeçalho `SDL_video.h` da própria distribuição baixada, não só a wiki, para confirmar os nomes exatos das macros e a assinatura de

`SDL_CreateWindowWithProperties` . A sequência que montamos foi `SDL_CreateProperties()` , depois um `SDL_Set*Property()` para cada propriedade, depois

`SDL_CreateWindowWithProperties()` e por fim `SDL_DestroyProperties()` .

Cálculo de endereço de pixel via `pitch` . Confirmando a preocupação da Etapa 1 (seção 7.3): o `pitch` de uma `SDL_Surface` não é necessariamente `largura * bytes_por_pixel` , por causa de alinhamento de linha. A solução foi sempre indexar como `pixels + y * surface->pitch + x * bytes_per_pixel` , nunca `y * surface->w * bytes_per_pixel` , e converter toda imagem carregada para um formato fixo (`SDL_PIXELFORMAT_RGBA32`) logo depois de `IMG_Load()` , usando `SDL_ConvertSurface()` . Isso tirou do resto do programa a necessidade de tratar formatos de pixel variados; a ideia já estava certa na Etapa 1 (seção 2.2-e), a implementação só confirmou que funciona.

Achatamento visual do histograma. Também previsto na Etapa 1 (seção 6, item 4): quando um valor de contagem fica muito mais alto que os demais, comum em imagens com fundo uniforme, o resto das barras some se a altura for normalizada linearmente pelo valor máximo. Resolvemos comprimindo a altura de cada barra em escala logarítmica ($\log(1 + \text{contagem}) / \log(1 + \text{contagem_máxima})$), o que preserva a visibilidade das barras baixas sem distorcer a ideia geral da distribuição.

Equalização e o `cdf_min` . O algoritmo segue a equalização clássica por função de distribuição acumulada (CDF): calcula o histograma, acumula em `cdf[i]` , encontra o menor valor de CDF diferente de zero (`cdf_min`) e mapeia cada intensidade `i` para $\text{round}((\text{cdf}[i] - \text{cdf_min}) / (\text{total_pixels} - \text{cdf_min}) * 255)$. Sem subtrair o `cdf_min` , imagens cujo histograma não começa em zero saem com a faixa de saída deslocada, sem usar o intervalo `[0, 255]` por completo. Também precisamos tratar o caso em que `total_pixels == cdf_min` (imagem de um único tom constante): nesse caso a divisão por zero é evitada mantendo a imagem como está.

Duas versões da imagem em memória. O requisito 5 pede reverter para a imagem original em escala de cinza sem recarregar o arquivo. Resolvemos mantendo `original_gray` (nunca liberada até o programa fechar) e `equalized_gray` (calculada uma vez, sob demanda, e guardada em cache) como duas `SDL_Surface*` separadas dentro do `AppState`, alternando qual delas vira a textura exibida através de `showing_equalized`.

Ambiente de teste sem sessão gráfica interativa. O ambiente onde escrevemos e compilamos o código não permitia testes interativos, cliques de mouse, verificação visual. Para não deixar a lógica de processamento de imagem sem nenhuma verificação, escrevemos um pequeno programa de teste isolado (não versionado, usado só durante o desenvolvimento) que chama direto `image_load`, `image_is_grayscale`, `image_to_grayscale`, `histogram_compute`, `histogram_equalize` e `IMG_SavePNG` sobre imagens de teste geradas com Python/Pillow, e imprime as estatísticas resultantes. Os resultados confirmaram: detecção correta de imagem colorida contra cinza, equalização aumentando o desvio padrão e esticando o intervalo de intensidades para `[0, 255]`, gravação de PNG íntegra (reaberta e conferida com Pillow). Essa verificação não substitui o teste visual da interface, que aconteceu depois, direto na máquina do João (ver seção 11).

6. Itens "confortáveis" na Etapa 1 que precisaram de referência extra ou IA

Comparando com a tabela da seção 6 do relatório da Etapa 1:

Carregamento de imagem (item 1) continuou confortável: a API `IMG_Load()` mais `SDL_GetError()` funcionou como esperado, sem surpresas.

Detecção de cor e conversão para escala de cinza (item 2) tinha lógica simples (percorrer pixels, comparar R/G/B, aplicar a fórmula), mas precisou de leitura adicional de `SDL_GetPixelFormatDetails` e `SDL_GetRGBA / SDL_MapRGBA` para acessar os pixels corretamente na API do SDL3, diferente do SDL2, que a Etapa 1 já sinalizava como mudança relevante. Não é um item que a documentação da SDL3 sozinha resolvesse sem cruzar `SDL_pixels.h` com `SDL_surface.h`.

Janela principal 1024x768 centralizada (item 3, parte principal) se manteve confortável: `SDL_CreateWindow` com `SDL_SetWindowPosition` e `SDL_WINDOWPOS_CENTERED` funcionaram como

documentado.

Cálculo do histograma, média e desvio padrão (item 4, parte estatística) se manteve confortável, são fórmulas padrão de estatística sobre o vetor de 256 posições.

Botões com primitivas e estados visuais (item 5, parte de UI) se manteve confortável, usando só `SDL_RenderFillRect`, `SDL_RenderRect` e teste de ponto dentro de retângulo.

Salvar imagem com a tecla S (item 7) se manteve confortável; `IMG_SavePNG()` mais `SDL_GetPathInfo()` para checar se o arquivo já existia resolveram o item sem pesquisa adicional.

Resumindo: os itens que a Etapa 1 já classificava como confortáveis continuaram confortáveis na prática. A única exceção pontual foi o acesso a pixels em formato RGBA32 (item 2), que exigiu confirmar assinaturas de função direto no cabeçalho da biblioteca em vez de só na documentação em prosa.

7. Itens que a Etapa 1 apontou como "precisa pesquisar": as soluções ajudaram a entender melhor?

Janela secundária como janela filha (seção 7.1 da Etapa 1): sim. A implementação confirmou que, no SDL3, "janela filha" é uma propriedade de criação (`SDL_PROP_WINDOW_CREATE_PARENT_POINTER`), não uma flag do `SDL_CreateWindow()` clássico, e que múltiplas janelas exigem distinguir eventos pelo campo `windowID` de cada `SDL_Event` (é assim que `app_handle_event()` roteia cliques de mouse só para a janela secundária). Isso ficou claro e é reaproveitável em qualquer programa SDL3 com várias janelas.

Equalização de histograma (seção 7.2): sim, e foi o item que mais aprofundou o entendimento teórico. Implementar a CDF na prática deixou claro por que a equalização é diferente de um simples alargamento linear de contraste: ela redistribui os pixels de acordo com a frequência observada, não só estica os valores mínimo e máximo. O tratamento do `cdf_min` também mostrou um detalhe que a descrição textual do algoritmo às vezes deixa de fora: sem ele, a equalização desloca a imagem para fora da faixa `[0, 255]` quando o histograma não começa em zero.

Manipulação de pixels em formatos diferentes (seção 7.3): sim. Converter a superfície para RGBA32 logo no carregamento, como planejado, realmente simplificou o resto do

código inteiro: `image.c` e `histogram.c` trabalham assumindo esse único formato, sem precisar de lógica condicional por formato de pixel.

Posicionamento condicional da janela (seção 7.4): sim, e de um jeito mais simples do que imaginávamos. `SDL_GetDisplayForWindow()` mais `SDL_GetDisplayBounds()` bastam para obter a resolução do monitor onde a janela está, e a decisão entre `SDL_WINDOWPOS_CENTERED` e `(0, 0)` sai direto daí, sem precisar calcular coordenadas de centralização na mão.

Escolha e empacotamento da fonte (seção 7.5): sim, conferimos a licença da DejaVu Sans e copiamos o `LICENSE.txt` junto, exatamente como planejado.

8. Como a IA generativa ajudou no desenvolvimento

Usamos uma ferramenta de IA generativa como apoio na implementação e na compreensão dos itens mais técnicos do escopo, principalmente a tradução das mudanças de API do SDL2 para o SDL3 (já mapeadas na análise inicial), a estrutura dos módulos em C e a integração entre `SDL_image` e `SDL_ttf`. A ferramenta ajudou a escrever e organizar código a partir das especificações que definimos, e a explicar, quando precisava, o funcionamento de APIs pouco documentadas em material introdutório, como a janela filha e `SDL_GetPixelFormatDetails`.

As decisões do projeto, porém, foram nossas: os limiares de classificação de brilho e contraste (seção 6), o tamanho e a cor dos elementos da interface, a forma de normalizar o desenho do histograma, a escolha e a licença da fonte, e a arquitetura geral dos módulos (`image`, `histogram`, `button`, `text`, `app/main`) foram discutidas e definidas por nós antes de serem implementadas. A IA entrou como ferramenta de implementação e consulta, não como quem decide o projeto.

Isso significa que revisamos e entendemos o código-fonte antes da entrega, sem aceitar nada "às cegas", porque a nota também avalia entendimento do conteúdo da disciplina. A seção 11 mostra exatamente essa revisão como parte do processo.

9. Assuntos que o grupo precisa estudar e praticar mais

Manipulação de buffers de pixel em C na mão, endereçamento via pitch, acesso a canais individuais: a lógica foi implementada e testada isoladamente, mas valeria a pena praticar mais esse padrão sem depender de IA.

Gerenciamento de múltiplas janelas e roteamento de eventos por `windowID` em SDL: é um padrão que não aparece nos exemplos de janela única, os mais comuns em material introdutório.

Equalização de histograma e suas variantes (alargamento de contraste, correção gama): a implementação cobre a equalização clássica, mas ainda falta entender formalmente as diferenças matemáticas entre as três técnicas, como já cogitávamos na Etapa 1. Vale revisitar o material da disciplina sobre transformações de intensidade.

Testes de interface gráfica sem sessão interativa disponível: precisamos de um fluxo de teste manual mais sistemático (checklist de cliques, redimensionamentos, estados de botão) para rodar antes de cada entrega, já que nem sempre dá para automatizar esse tipo de verificação.

Ferramentas de linha de comando para gerar PDF: a geração do PDF deste relatório exigiu montar uma solução alternativa (Markdown para HTML, depois impressão via navegador headless) por falta de `pandoc` /LaTeX no ambiente. Vale ter uma ferramenta de conversão instalada nas próprias máquinas, não só no ambiente usado durante o desenvolvimento.

10. Funcionalidades adicionais sugeridas (usabilidade e qualidade de vida)

Com 2 integrantes no grupo, o enunciado pede pelo menos 4 funcionalidades. Aqui está nossa análise sobre o software desenvolvido:

1. Desfazer/refazer (undo/redo) de operações. Hoje o programa só guarda duas versões da imagem, original em cinza e equalizada. Um histórico de operações com `Ctrl+Z` / `Ctrl+Y` tornaria natural encadear vários ajustes (equalizar, aplicar outro filtro, desfazer só o último passo) sem perder trabalho.
2. Ajuste manual de brilho e contraste, com sliders na janela secundária, complementando a equalização automática. A equalização é tudo ou nada; quem usa ferramentas reais de imagem costuma esperar poder ajustar aos poucos e ver o resultado em tempo real.
3. Arrastar e soltar (drag-and-drop) de arquivo de imagem na janela principal, como alternativa a passar o caminho por linha de comando. Reduz o atrito para quem só quer testar o programa rapidamente.
4. Escolha do formato ou caminho de saída ao salvar, em vez de sempre gravar `output_image.png` no diretório atual. Um diálogo simples, ou pelo menos um segundo argumento opcional de caminho de saída, evitaria sobrescrever arquivos por engano e permitiria salvar em JPG ou BMP além de PNG (o `SDL_image` já suporta `IMG_SaveJPG`, por exemplo).
5. Comparação lado a lado, antes e depois da equalização, em vez de substituir a imagem exibida. Ajudaria a avaliar visualmente o efeito da equalização sem ficar clicando repetidas vezes.
6. Zoom e rolagem (pan) na imagem exibida, para imagens muito grandes ou muito detalhadas, onde nem a resolução original nem o redimensionamento para 1024x768 são ideais para inspecionar uma região específica.

Já implementamos nesta etapa, embora isso não entre na contagem acima, os atalhos de teclado `E` (equalizar/reverter) e `R` (alternar resolução), equivalentes aos botões correspondentes.

11. Estado da entrega

A implementação e este relatório contaram com apoio de IA generativa, conforme registrado na seção 8.

O teste visual interativo completo (janela principal no tamanho e posição corretos, janela secundária realmente associada à principal, histograma proporcional, botões trocando de cor no hover e de texto ao alternar estado, equalização e alternância de

resolução atualizando a tela de fato, tecla `s` salvando o arquivo esperado, atalhos `E` e `R` funcionando) foi feito na máquina do João antes da entrega final.

A compilação no WSL Ubuntu com gcc 15.2.0 não pôde ser testada de ponta a ponta, porque nosso ambiente de desenvolvimento é Windows (seção 4 da Etapa 1), sem WSL instalado. O caminho de build para Linux, com o `Makefile` usando `pkg-config` para achar a SDL3 instalada no sistema em vez dos pacotes `devel-mingw` usados no Windows, está documentado no `README.md` e segue o processo oficial de instalação da SDL3 em Linux, mas recomendamos compilar numa máquina Ubuntu real (ou WSL) antes da entrega para confirmar.

Os limiares de classificação de brilho e contraste, o tamanho da janela secundária (360x420) e os tons exatos de azul dos botões foram decisões de projeto tomadas durante a implementação, documentadas nos comentários do código-fonte e no `README.md`.

12. Referências

- Enunciado do Projeto 1 (Proj1), Computação Visual, Prof. André Kishimoto, 2026.2.
- Relatório da Etapa 1 deste mesmo grupo (`relatorio/RELATORIO_ETAPA1.md`).
- Documentação oficial da SDL: <https://wiki.libsdl.org/SDL3/>
- Cabeçalhos oficiais das bibliotecas baixadas (SDL3 3.4.16, SDL3_image 3.4.6, SDL3_ttf 3.2.2), consultados diretamente para confirmar assinaturas de função durante a implementação.
- Fonte DejaVu Sans: <https://dejavu-fonts.github.io/> , licença em `assets/fonts/LICENSE.txt` .